

```
import numpy as np
```

vectorization means performing operations on an entire array at once without writing explicit loops

```
# Treditional Python Approch
array = np.array([1,2,3,4])
result = []
for i in range(len(array)):
    result.append(array[i]+10)
print(result)
```

```
[np.int64(11), np.int64(12), np.int64(13), np.int64(14)]
```

```
arr1 = np.array([1,2,3,4])
arr2 = np.array([5,6,7,8])
result = []
for i in range(len(arr1)):
    result.append(arr1[i] + arr2[i])
print(result)
```

```
[np.int64(6), np.int64(8), np.int64(10), np.int64(12)]
```

```
arr1 = np.array([1,2,3,4])
arr2 = np.array([5,6,7,8])
result = arr1+arr2
print(result)
```

```
[ 6  8 10 12]
```

```
a = np.array([1,2,3,4,5])
result = []
for i in range(len(a)):
    result.append(a[i]*10)
print(result)
```

```
[np.int64(10), np.int64(20), np.int64(30), np.int64(40), np.int64(50)]
```

```
result = a*10  
print(result)
```

```
[10 20 30 40 50]
```

Broadcasting : it is allows to perform any arithmetic operations on arrays with different shapes

```
arr1 = np.array([  
    [1,2,3],  
    [4,5,6]  
)  
arr2 = np.array([10,20,30])  
  
print(arr1+arr2)
```

```
[[11 22 33]  
 [14 25 36]]
```

Rows Brodcasting

- One Dimension is 1
- Dimensions are equal
- Dimension doest not exists

```
arr1 = np.array([  
    [1,2,3],  
    [4,5,6]  
)  
arr2 = np.array([[10],[20]])  
print(arr1+arr2)
```

```
[[11 12 13]  
 [24 25 26]]
```

Aggregate functions

- sum
- mean
- max
- min
- std
- var

```
arr = np.array([10,20,30,40])
print("Sum of all elements : ",np.sum(arr))
print("Mean of all elements : ",np.mean(arr))
print("Max from all elements : ",np.max(arr))
print("Min from all elements : ",np.min(arr))
print("Standard Deviation : ",np.std(arr))
print("Variance : ",np.var(arr))
```

```
Sum of all elements : 100
Mean of all elements : 25.0
Max from all elements : 40
Min from all elements : 10
Standard Deviation : 11.180339887498949
Variance : 125.0
```

linear algebra :almost every machine learning algo can be written as

$Y = MX + b$

- X : input features
- M : Weights
- b : Bias
- Y : output

```
# Scalar ,Vectors, Matrices and Tensors
```

```
# A single number
x = 5
print(x)
```

```
5
```

```
# collection of numbers in a one dimensions
marks = np.array([10,20,30])
print(marks)
```

```
[10 20 30]
```

```
# 2-D collection of numbers
m1 = np.array(
    [
        [1, 2]
```

```
        [1,2],  
        [3,4]  
    ]  
)  
m2 = np.array(  
    [  
        [5,6],  
        [7,8]  
    ]  
)
```

```
print(np.dot(m1,m2))
```

```
[[[19 22]]  
 [[43 50]]]
```

```
print(m1 @ m2)
```

```
[[[19 22]  
 [43 50]]]
```

```
print(m1.T)
```

```
[[1 3]  
 [2 4]]
```

```
print(np.linalg.inv(m1))
```

```
[[-2.   1. ]  
 [ 1.5 -0.5]]
```

```
A = np.array(  
    [1,2],  
    [2,4],  
    [2,6]
```

```
] )
```

```
print(np.linalg.matrix_rank(A))
```

```
2
```

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
marks =[20,33,29,28,38]  
hours =[6 , 5, 6, 8,5]
```

```
plt.plot(marks,hours,marker="o",color="red")  
plt.show()
```



